

面向量子布尔函数 Oracle 综合的资源约束神经蒙特卡洛树搜索 与强化学习预算控制

中文论文稿 v40: 最新投稿证据与强化学习预算控制同步稿

2026 年 7 月 10 日

摘要

量子布尔函数 oracle 综合是 Grover 搜索、约束求解和量子密码分析等算法中的基础编译环节。现有方法通常从固定代数正规形、ESOP/LUT/XAG 逻辑网络、可逆模板或 CNOT 导向空间结构出发,但这些路线难以在 T-count、CNOT、深度、门数和辅助比特之间进行统一的资源约束搜索。本文把 bit-flip Boolean oracle synthesis 建模为 ANF/FPRM 项集合上的组合优化问题,并提出 Resource-NMCTS: 该逻辑层框架结合神经动作先验、蒙特卡洛树搜索、布尔环线性因子、frontier controller、Pareto archive、baseline-preserving guard, 以及一个判断是否值得追加 Pareto 搜索的上下文 bandit 拟合 Q 预算策略。SSHR 仅作为小规模 CNOT 导向基线,而不是本文搜索空间或主方法的组成部分。

在 $n \leq 6$ 的 177 个传统函数上, Pareto-Resource-NMCTS 相对 direct ANF 的平均 T-count 和加权资源分数分别降低 72.25% 和 67.80%; 相对 ESOP beam、10 s 单函数限时 weighted ESOP-MILP 和 SSHR-H 的 score 胜/负/平分别为 174/0/3、167/3/7 和 173/4/0。匹配搜索控制把 base/Pareto MCTS 相对 no-MCTS 的额外收益限定为 1.44%/4.69%, 避免把全部资源优势归因于树搜索。进一步地, 在与训练、验证集合完全不重叠的 160 个测试函数上, 拟合 Q 预算策略只对 71 个函数追加 Pareto 搜索, 保留 94.90% 的 Pareto 平均 score 增益, 并使保守实测搜索时间降低 13.13% (95% bootstrap CI: 6.80%–20.46%); 相对 base Resource-NMCTS 的平均 score 降低 3.48%, 相对 always-Pareto 的平均 regret 为 0.51%。

外部逻辑与可逆综合 probe 继续显示对 ROS-style LUT、mockturtle、Caterpillar、CirKit 和 legacy RevKit CLI 的加权 score 优势, 同时暴露 CNOT、depth 和 peak-ancilla trade-off。phase/Rz 分支中的 Affine-FPRM、宽预算搜索和 learned shortlist 均通过 up-to-global-phase 验证, 但仍属于逻辑 phase proxy。当前证据包含 15,774 条符号、phase-search 和 policy-pruning 验证行, 以及 $n = 21\text{--}30$ 上 700/700 条完整 truth-table bridge 行。本文据此只主张匹配逻辑层任务上的低 T-count、低加权资源分数和可审计的质量–搜索开销控制, 不主张普遍 CNOT/depth/ancilla 支配、全局最优性、完整 ROS 复现或硬件映射优势。

关键词: 量子 oracle 综合; 布尔函数综合; 强化学习; 蒙特卡洛树搜索; ANF; FPRM; 资源约束编译

1 引言

给定布尔函数 $f : \{0, 1\}^n \rightarrow \{0, 1\}$, Boolean oracle synthesis 需要生成实现

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle \quad (1)$$

的可逆逻辑线路。这个任务看似只是把经典逻辑嵌入量子程序，但在 fault-tolerant 或资源受限量子计算场景中，布尔表示、计算/反计算顺序、辅助比特复用和相位实现方式都会显著改变 T-count、CNOT、深度和辅助比特需求。因此，oracle synthesis 不是一个单步翻译问题，而是一个带有多资源目标和语义约束的离散搜索问题。

已有工作从多个方向处理这个问题。ESOP、XAG、LUT 和 BDD 类方法通过经典逻辑网络降低乘法复杂度或局部函数成本 [6, 7, 8]; RevKit 和 mockturtle 提供了量子/经典逻辑综合工具链入口 [9, 10, 11]; SSHR 则利用 hypercube 中 paralleloptope 空间结构进行 small-scale CNOT-oriented synthesis[13]。这些方法给出了重要基线，但它们通常把候选结构或目标指标固定在某一类表示中。对于同时关注 T-count、CNOT、depth、gate count 和 peak ancilla 的逻辑层 oracle 综合，固定表示不一定能覆盖足够的资源折中空间。

本文的出发点是把 oracle synthesis 的主要自由度显式写成搜索空间。我们用 ANF 与 FPRM 项集合表示状态，用布尔环因子、极性选择、线性奇偶因子和可逆仿射变换定义动作，用 MCTS 与神经先验搜索候选，用 Pareto archive 保留多资源折中，并用 guard 保证学习候选不破坏确定性 baseline。在得到语义验证通过的 base Resource-NMCTS 结果后，一个独立的上下文 bandit 拟合 Q 策略进一步决定是否值得追加昂贵的 Pareto 搜索。这个设计使 AI 只负责可审计的动作排序与预算控制；语义正确性始终由 ANF 展开、GF(2) 符号模拟和完整 真值表 逐点验证提供。

本文贡献如下：

- 提出 Resource-NMCTS，把量子布尔函数 oracle 综合建模为 ANF/FPRM 项集合上的资源约束搜索问题，而不是 SSHR paralleloptope 空间的局部扩展。
- 设计由布尔环线性因子、神经动作先验、MCTS、depth-frontier controller、Pareto archive 和 baseline-preserving guard 组成的逻辑层 synthesis pipeline，并给出强 no-MCTS、随机先验和随机深度对照。
- 将是否追加 Pareto-Resource-NMCTS 写成两动作上下文 bandit，在互斥的 320/160/160 train/validation/test 切分上学习并冻结 fitted-Q 预算策略，直接量化质量收益、搜索时间与 regret。
- 建立分层比较协议：同任务逻辑基线支撑头部 T-count/score 结论，SSHR 和公开最优值提供反例边界，外部工具链提供压力测试，搜索消融提供因果归因；baseline fairness ledger 为 9/9 通过，SSHR Table IV–VIII crosswalk 为 5/5 通过。
- 用 plan ANF 验证、emitted-circuit GF(2) 符号验证、phase up-to-global-phase 验证和 $n = 21-30$ 的 700/700 完整 真值表 bridge 构建分层正确性证据链。

a Resource-constrained neural MCTS oracle synthesis remains verifiable at every layer.

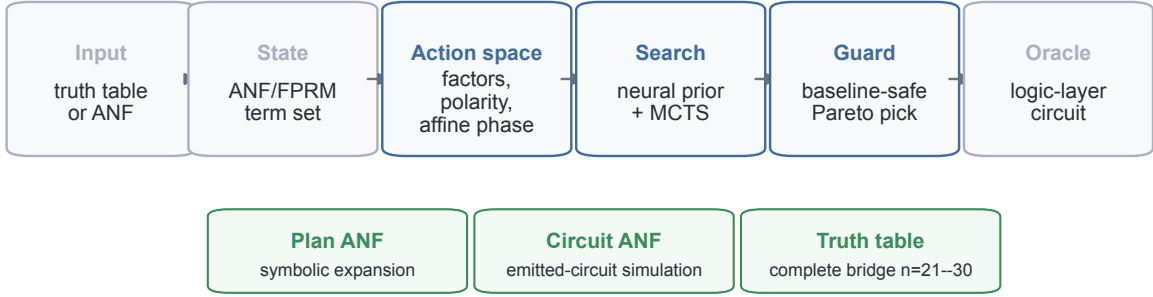


图 1: **Resource-NMCTS** 的可验证搜索 **pipeline**。输入可以是完整 真值表 或 ANF 项集合；动作空间包含布尔环因子、极性、线性奇偶因子和 affine phase 分支；搜索由神经先验、MCTS 和 frontier policy 控制；最终输出在 plan、emitted circuit 和完整 真值表 层进行验证。Source data: fig1_pipeline.csv.

2 相关工作

2.1 布尔表示与 oracle 综合

低 T-count oracle 综合长期依赖合适的布尔中间表示。BDD-based reversible synthesis 以符号决策图处理大函数 [6]；ROS 把 oracle synthesis 写成 resource-constrained LUT mapping，并强调 LUT 分解和后续可逆实现的资源耦合 [7]；XAG 将 XOR 与 AND 结构分离，使 AND 节点数成为非 Clifford 成本的重要 proxy [8]。ABC 提供成熟的 AIG/XAG/LUT/ESOP 逻辑优化流程 [1]，mockturtle、Caterpillar 与 CirKit 则分别提供可复用的逻辑网络、可逆 pebbling/API 和 shell 工具链入口 [9, 11, 2, 12]。这些路线说明，经典逻辑表示、垃圾管理和实现策略会共同改变量子 oracle 资源。

本文与上述工作共享“布尔表示决定量子资源”的前提，但不把搜索限制在某一种固定网络中。我们的状态是 ANF/FPRM 项集合，动作是可展开回 $GF(2)$ 多项式的代数分解。这样做牺牲了部分已有工具链的成熟优化，但换来了更直接的语义验证和更自然的多资源候选组合。

2.2 SSHR 与小规模空间结构方法

SSHR 利用 Boolean hypercube 中 parallelotope 结构生成 MCT block，并以 CNOT reduction 为主要目标 [13]。它适合本文作为 small-scale CNOT-oriented baseline，因为它关注小规模布尔函数、CNOT 计数和空间结构候选。本文不把 SSHR 当作主线方法：Resource-NMCTS 的状态、动作和验证均定义在 ANF/FPRM 项集合和代数搜索上。因此，本文可以在结果中承认 SSHR-H 的 CNOT 优势，同时主张自身在 T-count 和加权 score 上更强。

2.3 学习式搜索与线路综合

学习式搜索已经被用于经典电路设计、量子线路综合和图重写优化。ShortCircuit 使用 AlphaZero 风格搜索处理经典布尔电路设计 [14]；Gumbel AlphaZero 被用于 Clifford+T unitary synthesis[15]；强化学习也被用于 ZX-calculus rewrite-rule selection[16]；MonteQ 使用 MCTS 处理量子线路合成中的动作排序 [17]。上下文 bandit、Q-learning 与 fitted Q iteration 则为离线日志中的两动作预算决策提供了直接方法基础 [3, 4, 5]。这些工作共同表明，离散综合问题的动作空间和可选计算预算通常过大，需要学习策略或树搜索分配有限资源。

本文的区别在于，学习策略不直接写门序列。动作先验只对可验证的代数动作排序；fitted-Q controller 只决定是否调用额外 Pareto 搜索；正确性始终来自代数展开和 circuit-level symbolic simulation。因而本文可以分别量化表示空间、MCTS、learned prior、frontier controller 和强化学习预算策略的贡献，而不把模型预测与线路语义混为一谈。

3 问题定义与资源模型

本文考虑 bit-flip Boolean oracle。输入函数可以表示为完整 真值表，也可以表示为 square-free ANF

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \quad (2)$$

其中 $\mathcal{M}$ 是 monomial 集合。输出线路使用 X、CNOT 和多控 Toffoli (MCT) 抽象门，并在逻辑层估计 T-count、CNOT、depth、gate count 与 peak ancilla。

搜索排序采用统一加权分数

$$\text{score} = T + 0.04 \text{ CNOT} + 0.015 \text{ depth} + 0.01 \text{ gates} + 2 \text{ ancilla}. \quad (3)$$

这个分数只用于逻辑层候选排序，并不代表真实设备运行时间或物理错误率。因此，本文同时报告 T、CNOT、depth 和 ancilla，避免单一 score 掩盖 trade-off。

验证分为三层。第一层在小规模函数和高维 bridge 中构造完整 真值表，逐点验证式 (1)。第二层把 factor plan 展开回 ANF 项集合，检查目标多项式是否一致。第三层对 emitted X/CNOT/MCT circuit 做 GF(2) 符号模拟，检查输出线多项式、输入线复原和辅助线清零。第三层不枚举 2^n 个输入，但验证的是最终 emitted circuit，而不只是搜索 plan。

4 方法

4.1 项集合状态与可验证动作

Resource-NMCTS 的状态是当前尚未综合的项集合 $\mathcal{M}$ 。一次动作选择一个可计算、可反算的代数结构，将 $\mathcal{M}$ 分解为 quotient 子问题和 rest 子问题，再递归生成 compute/action/uncompute 线路。候选动作包括公共单项式因子、FPRM 极性重写后的公共项、线性奇偶因子、布尔环线性因子和 root-action 扩展。动作被接受后，框架估计即时资源收益、子问题大小、辅助比特峰值和后续可分解性。

神经动作先验用于对候选排序, MCTS 在有限预算内探索候选组合, rollout 使用确定性资源估计和已有 baseline 候选。最终 portfolio 在多个候选线路中按式 (3) 选择最优者。这里 AI 的职责是搜索控制, 而不是语义生成; 任何候选线路都必须通过 ANF 或 circuit-level 验证。

4.2 从具体布尔函数到量子线路的完整实例

考虑五变量函数

$$f(x) = x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_1x_2x_3 \oplus x_0x_1x_4 \oplus x_2x_3x_4. \quad (4)$$

令输入编号为 $j = \sum_{i=0}^4 2^i x_i$, 则其 32 位真值表为 0xB008C880, on-set 为 $\{7, 11, 14, 15, 19, 28, 29, 31\}$, 其余 24 个输入均输出 0。该函数可改写为

$$f = x_0x_1(x_2 \oplus x_3 \oplus x_4) \oplus x_2x_3(x_1 \oplus x_4). \quad (5)$$

表 1 给出使用本文 logical-AND 资源模型、训练动作评分器、seed 7 和 128 次 MCTS simulation 得到的真实根节点轨迹。合并 prior 为 $p = h + 2.5s_{\text{NN}}$, 其中 h 是资源启发式分数, s_{NN} 是训练网络的输出。对 x_0x_1 , 神经网络以 $s_{\text{NN}} = 0.927$ 把 $h = 0.381$ 提升为 $p = 2.700$ 。estimated score 来自已访问动作的 MCTS Q 值或确定性 rollout。 x_0x_1 获得 96 次访问, x_2x_3 获得 32 次访问; 两种先后顺序在 rest 子问题中选择另一个因子后得到相同最终 score, 因此确定性 tie order 首先输出 x_0x_1 。重叠候选 x_1x_2 和 x_1x_3 的 rollout score 更高, 没有获得后续访问。

表 1: 式 (4) 的根节点搜索轨迹。粗体为首先输出的因子; Combined 为 $h + 2.5s_{\text{NN}}$, estimated score 越低越好。

Factor	Grouped monomials	Heur.	Neural	Combined	Visits	Est. score
x_0x_1	$x_0x_1x_2 \oplus x_0x_1x_3 \oplus x_0x_1x_4$	0.381	0.927	2.700	96	33.255
x_1x_2	$x_0x_1x_2 \oplus x_1x_2x_3$	0.238	0.274	0.923	0	37.605
x_1x_3	$x_0x_1x_3 \oplus x_1x_2x_3$	0.238	0.274	0.923	0	37.605
x_2x_3	$x_1x_2x_3 \oplus x_2x_3x_4$	0.238	0.274	0.923	32	33.255

完整 Resource-NMCTS portfolio 与独立运行的 neural-MCTS ANF 分支收敛到相同双因子 plan。图 2 对比 AND-direct ANF 和最终 factored circuit。Direct 线路对五个三阶单项式分别施加一次三控输出操作; 搜索线路先计算 $a_0 = x_0x_1$, 依次复用于 x_2, x_3, x_4 , 反计算后再用同一条辅助线计算 $a_0 = x_2x_3$, 并复用于 x_1, x_4 。

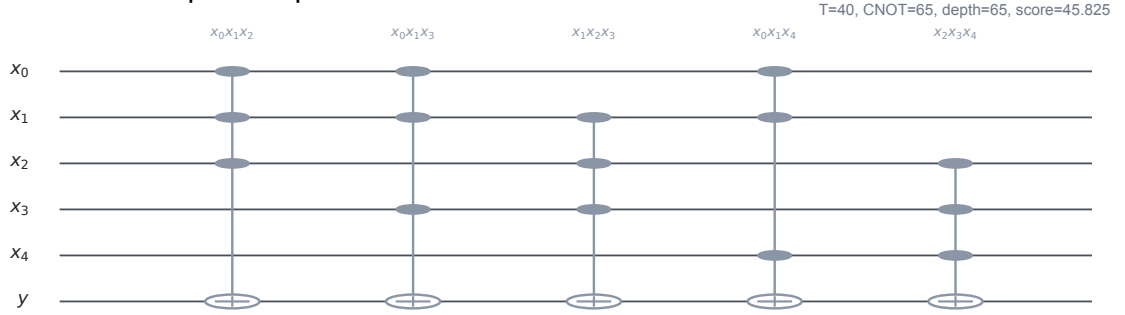
表 2: 具体实例的逻辑资源。Factor anc. 为显式复用因子线; peak ancilla 还包含分解 helper。Verified 为通过完整真值表检查的输入数。

Plan	T	CNOT	Depth	Gates	Factor anc.	Peak anc.	Score	Verified
AND-direct ANF	40	65	65	25	0	1	45.825	32/32
Resource-NMCTS	28	55	55	23	1	1	33.255	32/32

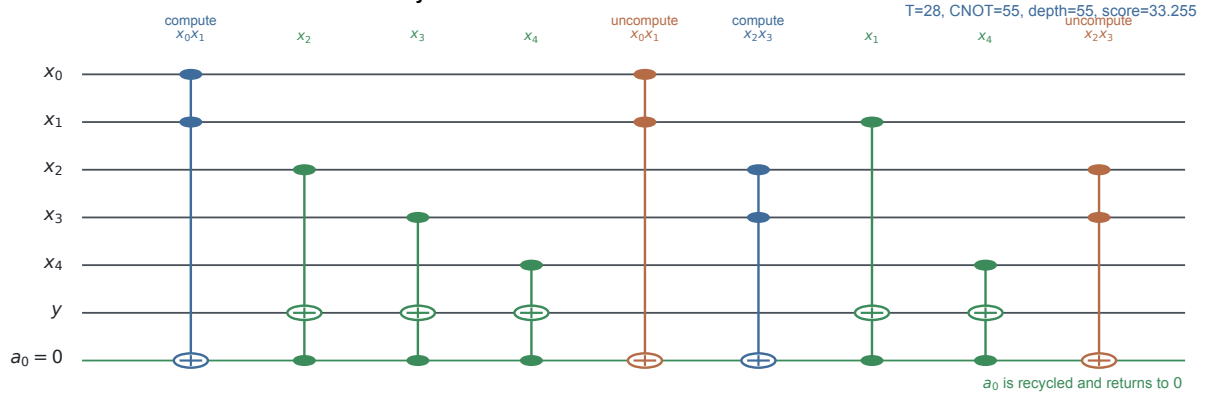
该计划把 T-count 从 40 降到 28 (30.00%), CNOT 和 depth 从 65 降到 55 (15.38%), gate count 从 25 降到 23 (8.00%), 加权 score 从 45.825 降到 33.255 (27.43%)。虽然增加了一条显式

$$f = x_0x_1(x_2 \oplus x_3 \oplus x_4) \oplus x_2x_3(x_1 \oplus x_4) \quad | \quad \text{truth table } 0xB008C880$$

a AND-direct ANF: five independent output actions



b Resource-NMCTS: two factors share one recycled line



The selected circuit passes plan expansion, emitted-circuit GF(2) simulation, and all 32 truth-table assignments.

图 2: **Resource-NMCTS** 的完整线路实例。(a) AND-direct ANF 对式 (4) 中每个单项式施加独立三控操作；(b) Resource-NMCTS 按式 (5) 执行 compute/reuse/uncompute, 并把复用辅助线恢复为 0。图中是通过验证的 X/CNOT/MCT 逻辑骨架；资源数字采用本文 logical-AND 口径, 其中合取计算使用 4T, 反计算使用 Clifford/measurement-assisted accounting。Source data: fig8_worked_example.csv.

factor line, 但 direct 三控操作在同一 logical-AND 分解模型下已经需要一个 helper, 因此两条线路的 peak ancilla 都为 1。正确性同时通过三层检查: factor plan 展开含 5 个节点且 0 mismatch; 9 个 emitted logical gate 的 GF(2) 模拟中输入、输出和辅助线 mismatch 均为 0; 完整真值表 32/32 通过。

这个实例用于解释核心因子搜索, 而不是证明 learned prior 在该简单函数上不可替代。它没有覆盖非 identity FPRM/affine 动作、Pareto 预算控制器、phase/Rz 分支或高维 frontier policy。

4.3 布尔环线性因子

传统线性因子筛选往往要求线性因子变量和 quotient 变量不相交。本文的 Boolean-ring screen 允许二者共享变量, 并在展开时使用 $x_i^2 = x_i$ 与 GF(2) 偶数项消去。例如

$$(x_i \oplus x_j)x_im = x_im \oplus x_ix_jm. \quad (6)$$

线路层面仍可用 CNOT 计算线性奇偶量, 再以该辅助量控制 quotient 子线路。该设计扩大了可搜索代数结构, 而不是单纯增加 beam width。

4.4 Frontier policy、Pareto archive 与 guard

本文把 AI 模块拆为三类。第一类是动作排序, 模型根据候选覆盖率、项数变化、变量覆盖密度、次数分布和即时资源估计对候选动作打分。第二类是结构选择, depth policy 判断当前项集合适合 single、depth-1 还是 depth-2 screen。第三类是 frontier 选择, depth-frontier policy 在 depth-2、depth-3 和 depth-4 screen 之间选择; quality policy 接近 all-depth frontier, cost-aware policy 用小幅 score 代价换取更低 plan time 和辅助线 lifetime area。

Guard 负责限制学习策略风险。若学习候选相对确定性 baseline 出现 score 退化, 则返回 baseline。Pareto archive 在多个候选中保留不同资源折中, 再由目标 score 选择最终输出。这个设计使算法可以报告“质量提升”“时间节省”和“质量-时间折中”三类贡献。

4.5 上下文 bandit 拟合 Q 的 Pareto 预算策略

Pareto-Resource-NMCTS 能改善线路资源, 但其搜索开销明显高于 base Resource-NMCTS。因此, 本文在已经得到语义验证通过的 base 结果后, 把“停止”或“追加 Pareto 搜索”建模为两动作上下文 bandit[3]。状态向量包含输入 ANF 的项数、次数和变量支持特征, 以及 base 线路的 score、T-count、CNOT、depth、gate count 与 peak ancilla。两个动作最终都返回经过相同语义验证器检查的线路, 学习模型不参与正确性判定。

设 base score 为 s_R 、Pareto score 为 s_P 、Pareto 时间为 t_P , 训练集 Pareto 时间中位数为 $\tilde{t}_P$, 则追加 Pareto 的日志回报定义为

$$r_{\text{run}} = \frac{s_R - s_P}{\max(s_R, 1)} - 0.004 \log_2 \left(1 + \frac{t_P}{\tilde{t}_P} \right), \quad r_{\text{stop}} = 0. \quad (7)$$

由于日志中同时存在 stop 与 run 两个动作的结果, 小型 fitted-Q 网络直接学习 run-action return, 遵循 Q-learning 与 fitted Q iteration 的批量价值学习思路 [4, 5]。策略使用 seed 42-43 的 320 个随机真值表函数训练, 在 160 个 seed-44 函数上选定阈值 0.60, 并冻结后在 160 个 seed-45 函数上测

试；三组精确 (n , truth table) 指纹完全互斥。这个模块属于强化学习式搜索预算控制，而不是端到端 deep RL 或 learned circuit generator。

5 实验设置

实验回答五个问题。第一，Resource-NMCTS 在完整 真值表 可验证的小规模函数上是否降低 T-count 和加权资源分数。第二，与同任务代数基线、SSHR、公开最优库、ROS-style LUT、mockturtle、Caterpillar、CirKit、RevKit CLI 和 RevKit phase/Rz probe 相比，结论分别能支持到哪一层。第三，affine search、MCTS、learned prior、frontier、guard、Pareto archive 和 fitted-Q budget policy 各自贡献多少。第四，资源权重变化、CNOT-only 目标和原始多资源非支配关系是否暴露反例。第五，语义验证能够可靠扩展到多大的逻辑实例。

传统核心切片包含 $n \leq 6$ 的 177 个函数。weighted ESOP-MILP 对每个函数设置 10 s 求解上限，报告该加权 ESOP 模型返回的可行 incumbent；它不是量子线路全局最优证书。外部逻辑网络覆盖 ABC/BDD、ROS-style LUT、mockturtle、Caterpillar 和 CirKit；RevKit CLI 使用完整 permutation 上的 exact-oracle reversible synthesis，RevKit Python `oracle_synth` 则单独作为 phase/Rz 边界。高维实验包括 $n = 20, 24, 28, 32, 40$ 项集、 $n = 48, 56, 64$ 超大规模符号压力切片，以及 $n = 21-30$ 的完整 真值表 bridge。所有配对比较只纳入名称匹配、语义检查成功且满足各自 timeout 契约的行。

预算策略实验与 177 函数头部表完全分离：seed 42–43 的 320 个随机真值表函数用于训练，seed 44 的 160 个函数用于选定阈值，seed 45 的 160 个函数只用于冻结策略测试。策略时间采用保守口径：选择 run 时同时计入已经支付的 base-Resource-NMCTS 时间与 Pareto-Resource-NMCTS 时间，即使当前 Pareto 实现内部会重复部分 base 候选。

表 3: 比较对象与结论层级。只有同任务核心行支撑头部 T-count/score 结论；其余行分别承担压力测试、反例、归因或边界作用。

层级	对比对象	可支持结论	明确排除
头部证据	Direct/AND-direct ANF、ESOP beam/MILP、ABC-ESOP/BDD	匹配 bit-flip oracle 上更低 T-count 与加权 score	普遍 CNOT、depth、ancilla、硬件或全局最优
外部压力	ROS-style LUT、mockturtle、Caterpillar、CirKit、RevKit CLI、ABC 网络	结论并非只来自本地手写弱基线	完整 ROS、路由、原生门或硬件复现
反例边界	SSHR CNOT、CirKit depth、Caterpillar CNOT、RevKit line/ancilla、STG optima	其他方法保留真实单资源优势	完整 Pareto 支配或“击败全部基线”
搜索归因	no-MCTS、beam、随机先验/深度、sparse guard、fitted-Q	AI/MCTS 提供有界质量、剪枝和预算收益	deep RL 单独解释全部 67.8% 改善
规模验证	$n = 20-64$ 符号压力与 $n = 21-30$ 完整 bridge	在声明的验证包内保持逻辑正确	高维全函数穷举或最优性证明
Phase 迁移	RevKit <code>oracle_synth</code> 、ANF/FPRM phase parity 与 affine shortlist	搜索框架可迁移到逻辑 phase/Rz proxy	最终 Clifford+T 旋转综合或 bit-flip 头部结果

表 4: 实验论据阶梯。原始 39,849 行中 39,368 行通过其所属层级的验证；不同层级的验证强度不可互换。

论据层级	范围	实例	验证行	作用	边界
精确同任务核心	$n = 3-6$	177	3819	完整真值表头部比较	小规模精确，不是大规模最优证书
公开最优值反例	$n = 4-5$	54	270	暴露更强 <code>tiny-function optimum</code>	负向边界，不是本文胜利行
外部工具压力	$n = 3-18$	373	968	<code>export/readback</code> 与逻辑网络检查	不是硬件映射或完整可逆后端
可逆/相位边界	$n = 3-6$	181	10109	<code>permutation, phase</code> 与 <code>coarse smoke</code>	不是最终近似 <code>Clifford+T</code> 比较
完整高维 <code>bridge</code>	$n = 21-30$	60	880	完整真值表与 <code>emitted-circuit</code> 检查	只覆盖生成式窄切片
超大规模符号压力	$n = 20-64$	384	7392	ANF 与 <code>emitted-circuit</code> 符号验证	不等于高维真值表穷举
AI/搜索控制归因	$n = 3-40$	385	15930	同预算、 <code>held-out</code> 与独立 <code>seed</code> 对照	不说明深度学习单独导致全部收益

6 结果

6.1 传统函数上的多资源优势

图 3 和表 5 给出 $n \leq 6$ 传统函数切片上的平均资源。Pareto-Resource-NMCTS 相对 direct ANF 的平均 T-count 和 score 分别降低 72.25% 和 67.80%；相对 10 s 单函数限时 weighted ESOP-MILP 的平均 T-count 和 score 分别降低 32.77% 和 29.84%。对 SSHR-H，score 胜/负/平为 173/4/0，而 T-count 为 173/0/4；SSHR-H 的平均 CNOT 仍最低。配对 sign test、bootstrap effect interval 和多权重敏感性审计均保留了 T/score 主结论，但 CNOT-only 重新搜索继续显示 SSHR 是强反例。因此，本文主张的是匹配逻辑层任务上的低 T-count 与低加权 score，而不是 CNOT-only、全指标或全局最优支配。

b

Pareto-Resource-NMCTS lowers T-count and weighted score, while SSHR-H remains a CNOT-oriented baseline.

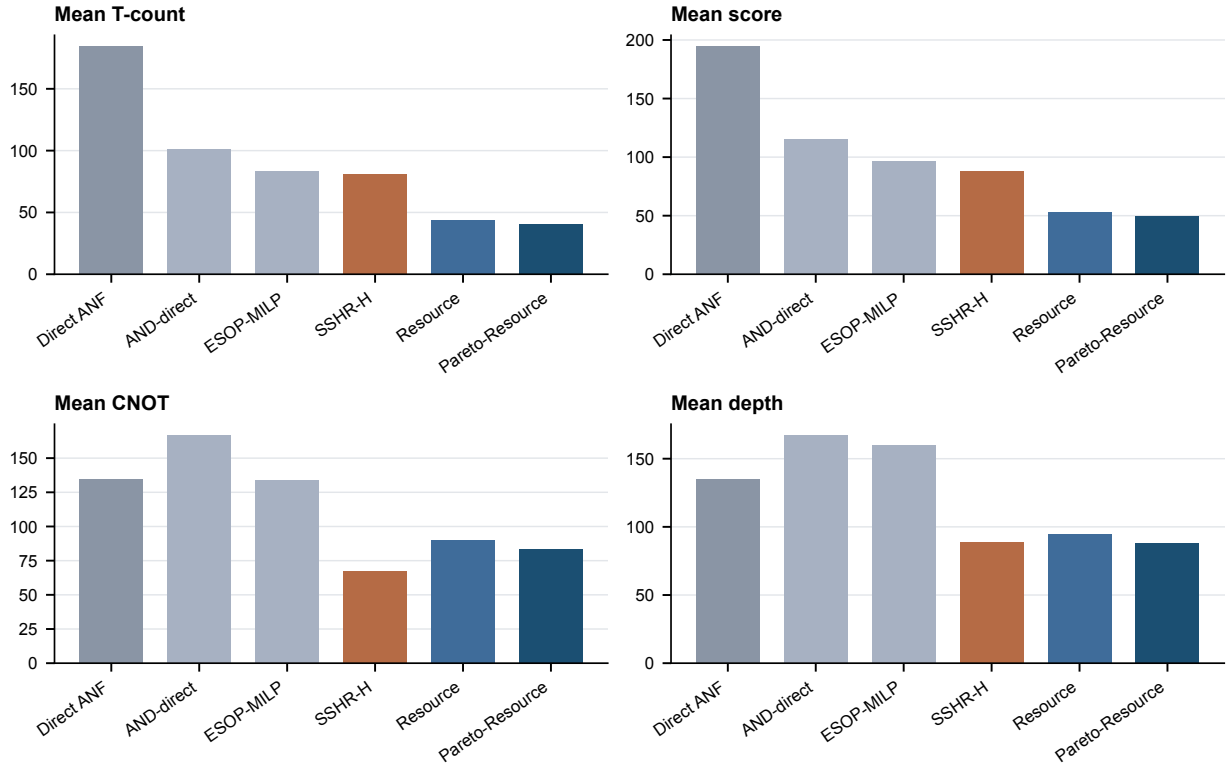


图 3: 传统函数上的平均资源。Pareto-Resource-NMCTS 在 T-count 和 weighted score 上最低; SSHR-H 仍是强 CNOT-oriented baseline。Source data: fig2_traditional_resources.csv.

表 5: $n \leq 6$ 传统函数切片上的平均逻辑资源。

方法	平均 T	平均 CNOT	平均 depth	平均 ancilla	平均 score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	67.6	88.7	1.36	88.2
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	40.4	83.0	88.0	2.03	49.6

6.2 外部 baseline 与可比边界

图 4 汇总外部和传统 baseline 的配对比较。Pareto-Resource-NMCTS 相对 ESOP beam 为 174/0/3, 平均 score 降低 36.09%; 相对 ESOP-MILP 为 167/3/7, 平均 score 降低 29.84%; 相对 SSHR-H 的 T-count 为 173/0/4, 平均降低 47.93%。ROS-style LUT proxy 覆盖 309 个函数, best- K proxy 比固定 $K = 4$ 更强, 但 Pareto-Resource-NMCTS 相对该 proxy 仍为 309/0/0。mockturtle probe 用官方 header 将 KLUT network 重综合为 XAG; 在传统 177 个函数上为 166/11/0, 在 $n = 14$ 的 64 个函数上为 64/0/0。

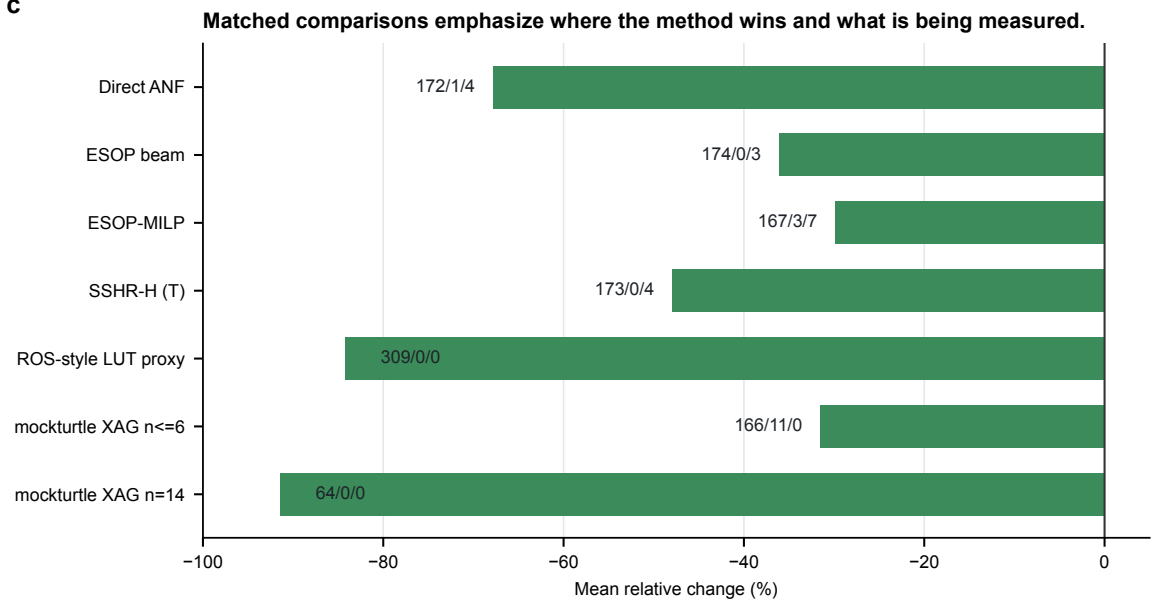


图 4: 配对基线比较。横轴为目标方法相对基线的平均变化；负值表示资源更低。数字标签给出胜/负/平；源数据见 `fig3_baseline_comparisons.csv`。

CirKit 3 shell AIG/MC probe 进一步补强外部工具链压力测试。该流程先用 ABC 把同一 BLIF benchmark 转成 AIGER, 再由 CirKit 执行 `cut_rewrite -a; resub -a; mccost -a`, 最后把 CirKit Verilog 输出交给 ABC 回读成 BLIF 并逐行 truth-table 验证。表 6 和表 7 显示, 传统集和 $n = 14$ 集均为全行 correct; Pareto-Resource-NMCTS 相对 CirKit AIG/MC 的 score 分别为 177/0/0 和 64/0/0, 平均降低 62.34% 和 94.46%。同时, depth 指标在传统集为 16/156/5, 在 $n = 14$ 为 14/50/0, 说明 CirKit AIG 在 depth proxy 上仍是强对手。

表 6: CirKit 3 shell AIG/MC probe: $n \leq 6$ 传统函数。负值表示目标方法资源更低。

Target vs CirKit AIG/MC	Items	W/L/T	Mean Δ
and_resource_nmcts	177	177/0/0	-60.84%
and_pareto_resource_nmcts	177	177/0/0	-62.34%
and_fprm_polarity_archive	177	177/0/0	-58.96%
and_direct_anf	177	124/53/0	-16.74%
direct_anf	177	46/131/0	+35.23%
and_esop_milp	177	150/27/0	-39.41%
sshr_h	177	164/13/0	-32.83%

表 7: CirKit 3 shell AIG/MC probe: $n = 14$ 高维函数。负值表示目标方法资源更低。

Target vs CirKit AIG/MC	Items	W/L/T	Mean Δ
and_resource_nmcts	64	64/0/0	-94.42%
and_profile_resource_nmcts	64	64/0/0	-94.42%
and_pareto_resource_nmcts	64	64/0/0	-94.46%
and_fprm_linear_pair	64	64/0/0	-94.27%
and_fprm_root_beam	64	64/0/0	-94.11%
and_direct_anf	64	64/0/0	-91.76%
direct_anf	64	64/0/0	-86.22%
and_fprm_greedy	64	64/0/0	-94.08%
and_affine_greedy	64	64/0/0	-94.08%

Legacy RevKit CLI exact-oracle probe 则从可逆综合角度补强对比。该流程把每个函数嵌入为 $(x, y) \mapsto (x, y \oplus f(x))$ 的完整 permutation, RevKit 用 `read_spec -p` 读入后运行 TBS、DBS 和 RMS 三个 synthesis flow, 并按 score 选择每个函数的 best-flow baseline。531/531 个直接 flow 行均返回, best-score portfolio 覆盖 177 个函数。表 8 显示, Pareto-Resource-NMCTS 相对 RevKit CLI best-score portfolio 的 score 为 173/0/4、平均降低 67.28%, T-count 为 173/0/4、平均降低 72.59%; 但 peak ancilla 为 0/169/8, 平均增加 153.11%。因此该基线强化了低 T/score 结论, 同时明确暴露辅助线 trade-off。

表 8: RevKit CLI portfolio: $n \leq 6$ 传统函数。负值表示目标方法资源更低。

Target vs RevKit CLI best-score	Items	W/L/T	Mean Δ
and_resource_nmcts	177	173/0/4	-66.32%
and_pareto_resource_nmcts	177	173/0/4	-67.28%
and_fprm_polarity_archive	177	173/0/4	-64.24%
and_direct_anf	177	167/6/4	-32.79%
direct_anf	177	87/86/4	+5.78%
and_esop_milp	177	172/1/4	-51.50%
sshr_h	177	170/7/0	+83.84%

Caterpillar XAG API probe 为外部工具链增加了另一组强反例。531/531 个策略行通过检查, Pareto-Resource-NMCTS 的 score 相对 Caterpillar 为 177/0/0、平均降低 47.94%; 但 CNOT 为 4/173/0、平均增加 195.18%。公开 STG 小函数最优库则给出更直接的负向控制: 在可比的 T-count optimum 行上, Pareto-Resource-NMCTS 为 0/40/14、平均高 84.26%。这些结果不是本文的“失败数据”, 而是限定结论不可越过的真实资源边界。

表 9: 核心定量答卷与最强反例。W/L/T 均以 Pareto-Resource-NMCTS 为第一方法；负百分比表示其资源更低。

比较对象	角色	有利结果	不利结果或边界
Direct ANF	同任务头部基线	score 172/1/4, -67.80%	不推出大规模或全局最优
10 s ESOP-MILP	同任务限时基线	score 167/3/7, -29.84%	仅为 ESOP 模型限时可行解
SSHR-H/SSHR-I	CNOT 导向反例	SSHR-H score 173/4/0, -41.06%	SSHR-H CNOT 43/128/6, 本文平均 +18.99%
Caterpillar API	外部 XAG probe	score 177/0/0, -47.94%	CNOT 4/173/0, 本文平均 +195.18%
CirKit / RevKit CLI	depth/ancilla 反例	score 分别 177/0/0, 173/0/4	CirKit depth 16/156/5; RevKit ancilla 0/169/8
公开 STG optima	tiny-function 最优值反例	无头部胜利主张	T optimum 0/40/14, 本文平均 +84.26%

baseline fairness ledger 对同任务、SSHR、外部逻辑网络、公开最优库、RevKit exact-oracle、phase/Rz、AI 控制、大规模验证和资源敏感性九个比较族逐一核对输入归一化、验证器、资源抽象和允许用途，当前 9/9 通过。该数字表示公平性契约完整，不表示本文在九类对象上全部获胜。

表 10: SSHR 原论文表格与当前复现范围的对照。

原论文表	当前覆盖	允许结论	明确未覆盖
Table IV	有界参考	作为原始门分解与 CNOT 导向背景	不声称完整重跑
Table V	177 个同函数 SSHR-H 行	同任务小函数 CNOT 参照	不等于全部随机实验复现
Table VI	177 个限时 SSHR-I CNOT 行及 72 个 $n \leq 4$ pilot	限时 CNOT 对照，保留 timeout 元数据	不是完整聚合或最优证书
Table VII	177 个限时 SSHR-I T 行及 72 个 $n \leq 4$ pilot	当前逻辑资源投影下的 T-oriented 参照	不覆盖全部 random/NPN 设置
Table VIII	$n = 3-8$ 候选数精确复现	49、257、1539、10299、75905、609441 全部一致	候选计数不能替代 Tables IV-VII 重跑

这些对比有意义，但需要严格限定。ROS-style LUT 是 proxy，不是官方 ROS 全流程；mockturtle probe 是 KLUT-to-XAG 逻辑网络比较；CirKit probe 是 AIG multiplicative-complexity 逻辑网络比较；RevKit CLI probe 输出 exact oracle permutation 上的可逆 synthesis 统计，但 CNOT/depth 仍由 Toffoli control distribution 派生，也不包含 ROS/SAT garbage management 或硬件 mapping。RevKit oracle_synth 返回 Rz-phase netlist，不能直接当作精确 T/Tdg 网络比较。本文据此只主张逻辑层资源优势，而不是工具链全流程或硬件后端优势。

6.3 搜索控制归因与强化学习预算策略

表 11 显示，最大算法增益首先来自可搜索的 affine/Boolean-ring 动作空间，而不是学习器本身。Full Affine-NMCTS 相对 fixed-coordinate MCTS 的平均 score 降低 14.82%，Pareto archive 相对单一 Resource-NMCTS 再降低 3.26%。在更强的确定性 no-MCTS portfolio 对照下，base Resource-NMCTS 为 54/0/123、平均 score 降低 1.44%，Pareto-Resource-NMCTS 为 106/0/71、平均降低 4.69%。同预算 learned prior 相对八次随机先验均值只有 17/8/152、平均降低 0.15%，且运行时间不利。因此，本文把神经先验写成有限质量信号，把 MCTS/Pareto 写成有界增量，而不把全部 67.8%

资源改善归因于深度强化学习。

表 11: 搜索贡献分解。负的 mean Δ score 表示目标方法更优。

Component	Dataset	Pairs	Score W/L/T	Mean Δ score
Affine greedy vs fixed-coordinate MCTS	ablation_affine	321	165/88/68	-12.12%
Neural refine over affine greedy	ablation_affine	322	65/0/257	-1.08%
Guarded Affine-NMCTS over no-guard	ablation_affine	322	88/0/234	-1.74%
Learned prior for Resource-NMCTS	traditional_resource	177	39/0/138	-1.10%
Pareto archive over Resource-NMCTS	traditional_resource	177	68/0/109	-3.26%
No-MCTS portfolio over heuristic-only	trad. ablation	177	54/0/123	-2.51%
Resource-NMCTS over no-MCTS portfolio	trad. ablation	177	54/0/123	-1.44%
Pareto Resource-NMCTS over no-MCTS portfolio	trad. ablation	177	106/0/71	-4.69%
Highdim no-MCTS portfolio over root beam	n14 ablation	16	14/0/2	-6.25%
Highdim no-MCTS portfolio over linear-pair	n14 ablation	16	14/0/2	-3.08%
Linear-pair guard vs root beam at n=14	highdim_resource	64	60/0/4	-3.00%
Recursive linear-pair guard vs root beam at n=15	n15 scale	32	30/0/2	-5.28%
Recursive linear-pair guard vs shallow linear-pair at n=16	n16 scale	24	23/0/1	-2.54%
Recursive linear-pair guard vs root beam at n=16	n16 scale	24	23/0/1	-4.31%
Baseline-preserving AI guard vs deterministic recursive guard at n=16	n16 scale	24	6/0/18	-0.05%
Fast linear-pair guard vs root beam at n=18	n18 stress	12	6/0/6	-1.91%
Resource-NMCTS vs fast linear-pair at n=18	n18 stress	12	12/0/0	-3.55%

高维 frontier 控制进一步把“质量”和“搜索开销”分开。稀疏 depth-2/4 frontier 在审计切片上与完整 depth-2/3/4 frontier 得到相同资源结果, 同时去除约四分之一到三分之一的 frontier evaluation time。learned depth-4 gate 在三组独立 seed、共 144 个 pair 上保持 0 false skip, 只运行 106 次 depth-4 evaluation, 减少 13.43% sparse-frontier evaluation time; 阈值扫描中的最佳 zero-false-skip 点节省 14.92%。分阶段 controller 相对 all-depth 的 score 只高 0.04%, staged planning time 降低 25.43%。这些控制器是当前生成式切片上的经验 operating point, 不证明 depth 3 或 depth 4 在所有实例上普遍无用。

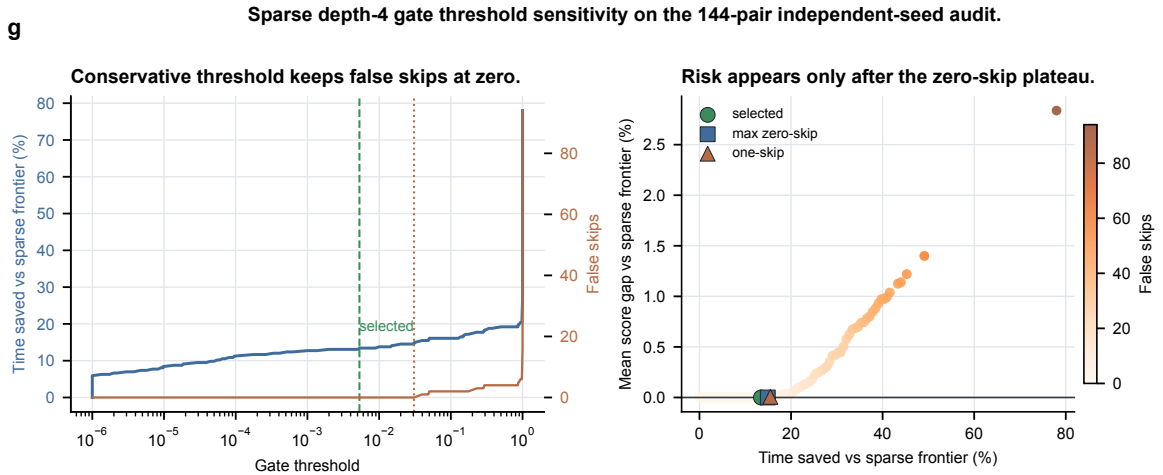


图 5: 稀疏 depth-4 gate 的阈值审计。选定阈值位于零错误跳过平台内, 本文采用保守工作点。

拟合 Q 的预算策略提供了当前最直接的强化学习证据。独立 seed-45 测试中, 策略对 71/160 个函数追加 Pareto 搜索、对 89/160 个函数停止, 因而把昂贵 Pareto 调用减少 55.6%。相对 base

Resource-NMCTS, score 为 56/0/104、平均降低 3.48%, T-count、CNOT 和 depth 分别降低 3.99%、3.84% 和 3.34%。相对 always-Pareto, 它保留 94.90% 的平均 score 增益并使保守时间降低 13.13%, 但仍有 13 个函数损失部分 Pareto 增益, 平均 score regret 为 0.506%; peak ancilla 相对 base 增加 4.48%。这支持“质量-搜索开销改善”, 不支持对 always-Pareto 的 score 支配。

表 12: 上下文 bandit fitted-Q 预算策略的独立测试结果。

阈值	Run/stop	对 Pareto W/L/T	对 base W/L/T	Score regret	时间与收益边界
0.60	71/89	0/13/147	56/0/104	+0.506%	时间 -13.13%; 保留 94.90% Pareto score gain

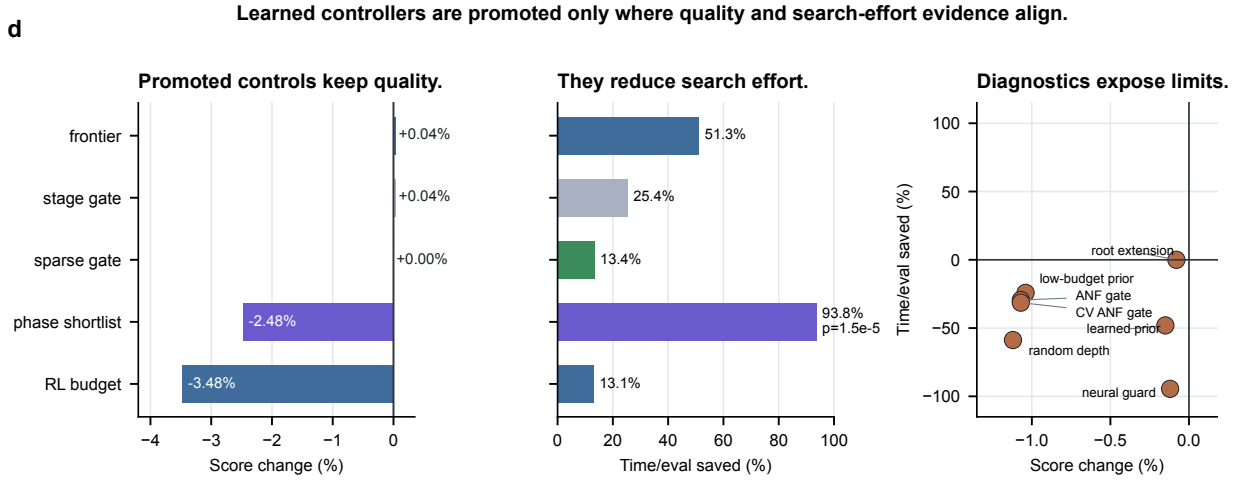


图 6: 学习与搜索控制证据汇总。 promoted controls 同时具备可验证的质量或开销收益; limited diagnostics 则保留小幅质量、运行时间反向或代理边界, 防止把所有 AI 模块写成同等主贡献。Source data: fig7_learned_control_summary.csv.

6.4 Phase/Rz 边界与 Affine-FPRM 搜索

RevKit oracle_synth 在 177 个传统函数上全部返回, 但 171/177 行含非 Clifford R_z , 总数为 9242。因此, 如果只按 lower-bound phase netlist 计分, 比较会低估 RevKit 输出在 Clifford+T 近似综合中的成本。为把该边界转化为本文内部可验证分支, 我们实现 phase-parity ANF emitter, 并进一步加入 fixed-polarity FPRM search 和 Affine-FPRM phase search。

图 7 显示, Affine-FPRM 在 $T/R_z = 30$ proxy 下相对 fixed-polarity FPRM 为 81/0/96, 平均 score 降低 2.51%; 相对 phase-parity ANF 为 85/0/92, 平均降低 2.98%; 相对 RevKit 为 177/0/0, 平均降低 65.50%。在 177 个函数中, 81 个选择非 identity 线性变换, 43 个选择非零极性。这个结果说明 phase/Rz 分支已经从成本敏感性讨论推进为可验证的代数搜索; 但它仍是 parity-gadget 级逻辑 emitter, 不包含近似旋转序列生成或硬件映射。

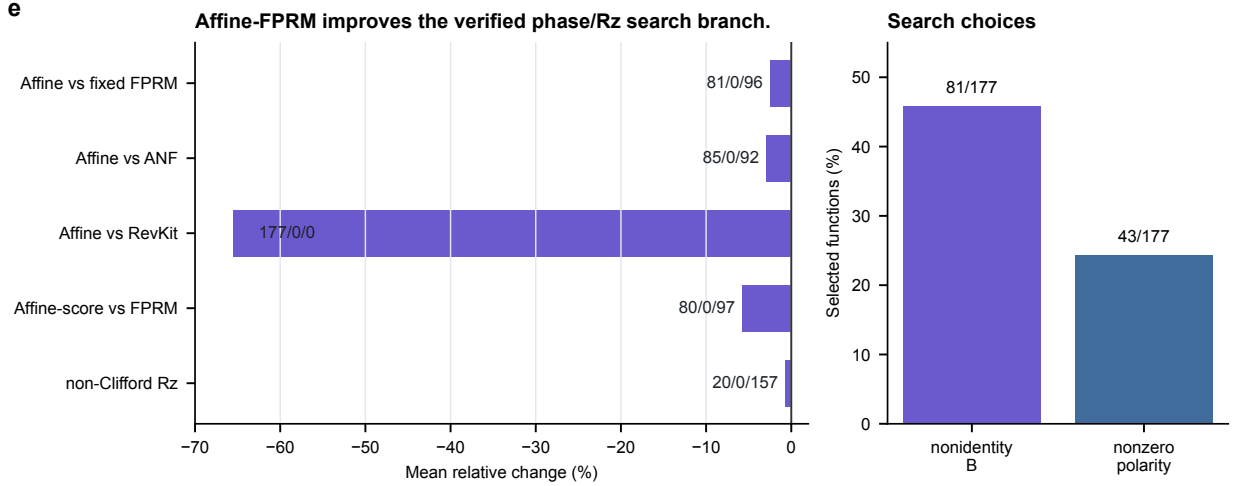


图 7: **Affine-FPRM** 相位搜索。左图为 Affine-FPRM 相对 fixed FPRM、ANF 和 RevKit proxy 的平均变化；右图为非 identity 线性变换和非零极性的选择频率；源数据见 `fig4_phase_affine.csv`。

表 13: Affine-FPRM phase-parity search 的配对比较。

Comparison	Items	W/L/T	Mean Δ
Affine-FPRM- $T/R_z = 30$ vs FPRM	177	81/0/96	-2.51%
Affine-FPRM- $T/R_z = 30$ vs ANF	177	85/0/92	-2.98%
Affine-FPRM- $T/R_z = 30$ vs RevKit	177	177/0/0	-65.50%
Affine-FPRM non-Clifford Rz vs FPRM	177	20/0/157	-0.73%
Affine-FPRM-score vs FPRM	177	80/0/97	-5.77%

为了检验该分支是否仍受搜索预算限制，本文进一步把 affine transform budget 从 32 扩展到 128，保持同一随机种子和确定性候选前缀，并输出独立结果文件。该设置使每个函数的候选 affine forms 均值从 1031.2 增加到 4124.9，并在 $n = 6$ 函数上进入更复杂的随机可逆线性变换，而不改变 phase-polynomial emitter 或验证方式。结果见表 14：对 rank metric 本身，wide128 相对 budget32 均无 score-loss；在 $T/R_z = 30$ 目标下为 43/0/134，平均 synth-score 再降低 0.60%。同时，total Rz、CNOT 和 depth 分别降低 2.39%、1.74% 和 1.93%。这说明 phase/Rz 分支的增益尚未耗尽，后续 learned phase/Rz policy 可以有明确的搜索空间承接，而不是只做固定极性枚举。

表 14: Affine-FPRM phase search 的预算扩展结果。比较对象为 transform-budget 128 相对 transform-budget 32 的同名 selected rows；负值表示 budget 128 更低。

Metric	Pairs	W/L/T	Mean Δ
score target	177	38/0/139	-1.59%
score+1/Rz target	177	40/0/137	-0.93%
$T/R_z = 30$ target	177	43/0/134	-0.60%
non-Clifford Rz	177	3/0/174	-0.29%
total Rz	177	35/2/140	-2.39%
CNOT	177	37/2/138	-1.74%
depth	177	41/2/134	-1.93%

在此基础上，本文把 phase candidate policy 更新为 rank-label 训练与 deterministic diversity rerank。训练仍使用 $n \leq 5$, held-out $n = 6$ 的 38 个函数只用于测试；每个函数从 transform-budget 128 生成的 8192 个 affine/polarity 候选中选择 shortlist，最终资源仍由 exact phase emitter 和 up-to-global-phase verifier 计算。diverse top-512 只精确计分 512/8192 个候选，相对 budget-32 为 17/0/21、平均 $T/R_z = 30$ synth-score 降低 2.477%，相对 wide-128 的均值 gap 仅 +0.003%，因而减少 93.75% 的 wide-search exact scoring。

同预算随机对照使该结论更加有界。diverse top-512 相对每函数八次 random shortlist 均值为 17/0/21、平均差 -0.012%，paired sign test 为 $p = 1.53 \times 10^{-5}$ ，且均值低于全部八个 random seed mean；但绝对差异仍很小。因此，该分支支持“learned pruned-search feasibility”和搜索预算前沿改善，不支持显著支配随机 shortlist 或 phase 全局最优。

旋转成本审计进一步把每个 non-Clifford R_z 按 $\lceil 3 \log_2(1/\epsilon) \rceil$ 计入 T/depth/gate proxy。在 $\epsilon = 10^{-6}$ 时，Affine-128 相对 RevKit `oracle_synth` 为 177/0/0、平均 score 降低 66.118%；diverse top-512 相对 budget-32 为 17/0/21、平均降低 2.381%，相对 wide-128 只高 0.002%。来源于实际 phase-search 输出的 20 个高频角在 $\epsilon = 0.125$ 的 coarse Clifford+T sequence smoke 中 20/20 通过，但在 $\epsilon = 0.05$ 时只有 5/20 通过。当前环境没有完整高精度 rotation backend，因此这些结果仍不是最终 Clifford+T 旋转综合。

表 15: Phase learned shortlist、精度敏感性与序列级边界。负值表示目标方法资源更低。

比较	主要结果	解释边界
diverse top-512 vs budget-32	17/0/21, score -2.477%	只精确计分 512/8192 个候选
diverse top-512 vs wide-128	均值 gap +0.003%	接近宽搜索，不是 score 支配
diverse top-512 vs random mean	17/0/21, $p = 1.53 \times 10^{-5}$	绝对幅度约 0.01%
rotation sequence smoke	20/20 at $\epsilon = 0.125$; 5/20 at 0.05	仅为 coarse sequence sanity check

6.5 高维验证与 bridge

图 8 汇总本稿主图采用的验证覆盖。 $n = 20-64$ item-level symbolic runs 为 6120/6120，其中 $n = 48, 56, 64$ 超大规模压力切片为 480/480；width probe 为 1512/1512，phase-search selected rows 为 531/531，phase-policy selected rows 为 7611/7611，合计 15,774 条符号、phase-search 和 policy-pruning 行通过所属验证器。完整 真值表 bridge 在 $n = 21-30$ 上为 700/700： $n = 21, 22$ 基础 bridge 120 行， $n = 23$ 基础、large-policy 与 cost-aware 三类共 160 行， $n = 24-30$ 各 60 行。全部 bridge 行同时通过完整 oracle、plan ANF 和 emitted-circuit 符号验证。超大规模行只证明生成式实例在符号验证包络内可运行，不等价于 2^n 真值表穷举。

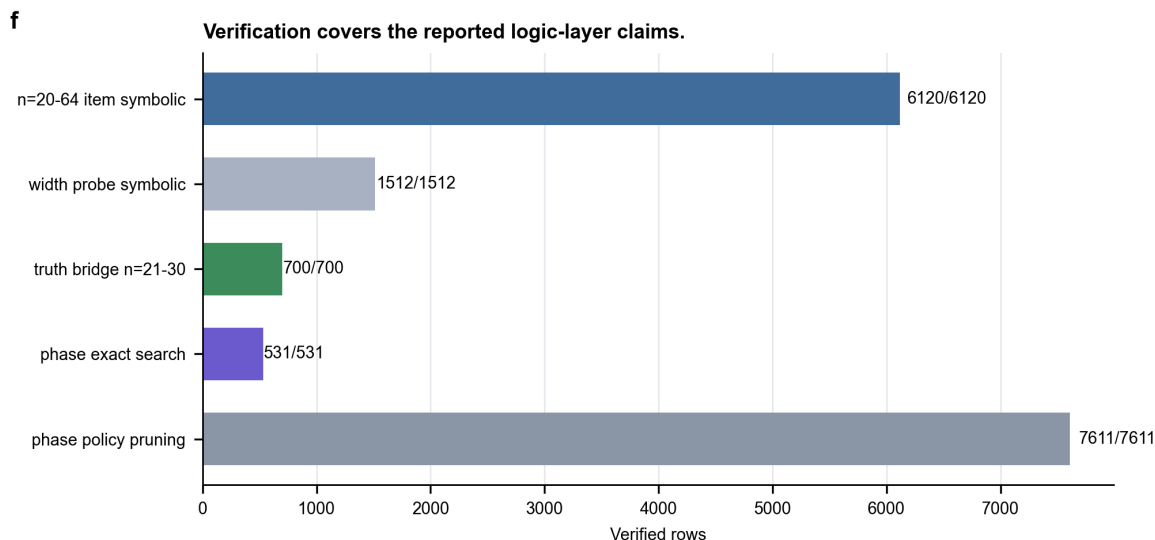


图 8: 验证覆盖。主图区分符号验证、phase/policy 验证和完整 真值表 bridge; 不同验证强度不作等价解释。Source data: fig5_validation.csv.

表 16: 高维验证与边界。

验证项	当前覆盖	边界
$n = 20-64$ item-level symbolic runs	6120/6120	含 480/480 条 $n = 48, 56, 64$ 压力行; 不是完整 真值表 枚举。
Width probe symbolic runs	1512/1512	证明盲目加宽不是主收益来源, 但仍是项集级验证。
$n = 21-30$ complete bridge	700/700	生成式小样本完整枚举, 不能外推为 $n = 31-64$ 全部函数。
Phase-search selected rows	531/531	up to global phase 验证, 不输出高精度旋转序列。
Phase-policy selected rows	7611/7611	learned shortlist 的候选级验证, 不是 phase 全局最优。
Rotation-sequence smoke	20/20 at $\epsilon = 0.125$	粗粒度单比特 sanity check; $\epsilon = 0.05$ 仅 5/20。

7 讨论

当前证据支持一个具体而有边界的结论: Resource-NMCTS 是一种逻辑层、资源约束的量子布尔函数 oracle 综合方法, 它在同函数、同验证器的 bit-flip benchmark 上降低 T-count 和加权资源分数。该结论不是“击败所有指标”。SSHR-H/SSHR-I 保留 CNOT 优势, CirKit 保留 depth 优势, Caterpillar 保留 CNOT 优势, RevKit CLI 在多数传统函数上使用更少辅助线, 公开 STG 小函数最优库也显著优于本文的 tiny-function T-count。把这些负向结果放入正文, 能够区分“强 T/score operating point”与“不存在的全资源支配”。

消融进一步校准了 AI 的作用。最大资源改善来自可搜索的 ANF/FPRM/Boolean-ring 动作空

间和受 guard 保护的 portfolio，而不是 learned prior 单独产生。强 no-MCTS 对照把 base/Pareto MCTS 的额外 score 收益限定为 1.44%/4.69%；随机先验对照则显示 generic bit-flip learned prior 的绝对收益较小且运行时间不利。上下文 bandit fitted-Q controller 是本文最明确的强化学习模块，但它只工作在 search-budget 层：独立测试显示，它以 0.506% 的平均 always-Pareto score regret 换取 13.13% 的保守搜索时间下降，并保留 94.90% 的 Pareto score gain。因而正确表述是“强化学习式质量-搜索开销控制”，而不是端到端 deep RL 生成线路或保证语义正确性。

高维结果主要回答可扩展性与正确性包络，而不是最优性。 $n = 20-64$ 的符号行验证了 plan 与 emitted circuit 的 GF(2) 语义， $n = 21-30$ 的 700/700 方法行进一步通过完整真值表检查。两类证据相互补充，但强度不同：符号验证可以扩展到更大 n ，完整真值表 bridge 则受指数增长限制，只覆盖生成式窄切片。实验论据阶梯因此把头部小函数、外部工具、可逆/phase、完整 bridge、超大规模符号压力和 AI 归因分开报告。

phase/Rz 分支同样保持有界。Affine-FPRM、wide-128 和 diverse shortlist 证明该代数搜索可以迁移到 up-to-global-phase 的逻辑 proxy，并在不同旋转成本假设下保持稳定趋势。random shortlist 的绝对差异仍很小，coarse rotation-sequence smoke 也没有形成高精度后端。由此不能把 RevKit oracle_synth 的 phase netlist 直接换算成精确 Clifford+T T-count，也不能声称已解决近似旋转综合。

其余限制均来自当前实验设计本身。ROS-style LUT 和 garbage-aware rows 仍是 proxy，而不是官方 ROS/SAT 全流程；mockturtle、Caterpillar、CirKit 和 ABC 行是逻辑网络或 API probe；RevKit CLI 是 exact-oracle reversible-synthesis counterpoint，但仍不包含硬件 mapping。全文没有 placement、routing、native-gate scheduling、噪声、magic-state factory 或容错开销模型。因此，所有性能结论都停留在声明的逻辑资源模型内。

8 结论

本文提出带强化学习预算控制的 Resource-NMCTS，把量子布尔函数 oracle 综合写成可验证 ANF/FPRM 动作上的资源约束搜索。匹配小函数实验支持相对 direct ANF、ESOP、10 s 限时 ESOP-MILP 和 SSHR-H 的低 T-count/低加权 score 结论；外部 ROS-style、mockturtle、Caterpillar、CirKit 与 RevKit probe 表明该结果并非只来自本地弱基线，同时也暴露 CNOT、depth 与 ancilla 反例。强 no-MCTS、随机先验、随机深度、sparse gate 和 fitted-Q 控制把表示空间、树搜索与学习预算的贡献分开。独立拟合 Q 测试进一步表明，可以在显式小幅 regret 下减少可选 Pareto 搜索开销。 $n = 20-64$ 符号验证和 $n = 21-30$ 完整真值表 bridge 支撑逻辑正确性，但不改变本文的逻辑层边界。后续工作应聚焦高精度 phase rotation synthesis、更加完整的外部工具链复现和独立硬件后端评估，而不是把当前结果外推为硬件级或全局最优性结论。

数据与代码可用性

本文使用的代码、实验驱动、模型、原始与汇总 CSV、manifest JSON、LaTeX 表格、图件 source data 和审计报告均维护在项目仓库的 resource_nmcts_experiment/ 目录。主要复现入口为 run_*.py、train_*.py 和 analyze_*.py；投稿图表位于 paper_latex/figures/submission_v36/

与 `paper_latex/tables/`。脚本 `rebuild_submission_package.sh` 可从现有实验产物重建论文面对的表图、审计、PDF 和 submission payload, 但不会重新运行全部原始 sweep、外部工具 probe 或神经训练。所有外部工具结果均附 manifest 级 provenance; 硬件 mapping、routing 和 magic-state factory 产物因不在本文范围内而未包含。

参考文献

- [1] R. K. Brayton and A. Mishchenko. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification*, LNCS 6174, pp. 24–40, 2010.
- [2] G. Meuli and collaborators. Caterpillar: Quantum circuit synthesis for Boolean functions. <https://github.com/gmeuli/caterpillar>, 2026.
- [3] L. Li, W. Chu, J. Langford, and R. E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of WWW*, pp. 661–670, 2010.
- [4] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.
- [5] D. Ernst, P. Geurts, and L. Wehenkel. Tree-based batch mode reinforcement learning. *Journal of Machine Learning Research*, 6:503–556, 2005.
- [6] R. Wille and R. Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of DAC*, pp. 270–275, 2009.
- [7] G. Meuli, M. Soeken, M. Roetteler, and G. De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.
- [8] G. Meuli, M. Soeken, and G. De Micheli. Xor-And-Inverter Graphs for quantum compilation. *npj Quantum Information*, 8:7, 2022.
- [9] M. Soeken, G. Meuli, E. Testa, H. Nada, M. Reymond, B. Dang, G. De Micheli, and A. Mishchenko. The EPFL logic synthesis libraries. *arXiv:1805.05121*, 2018.
- [10] M. Soeken and collaborators. RevKit: Python quantum compilation library. <https://github.com/msoeken/revkit>.
- [11] M. Soeken and collaborators. mockturtle: logic network optimization and synthesis. <https://mockturtle.readthedocs.io/>.
- [12] M. Soeken and collaborators. CirKit: logic synthesis and optimization framework. <https://github.com/msoeken/cirkit>.
- [13] Q. Zheng, Y. Xu, J. Zhang, Z. Su, and S. Zheng. CNOT oriented synthesis for small-scale Boolean functions using spatial structures of parallelotopes. In *Proceedings of ICCAD*, 2025.
- [14] D. Tsaras, A. Grosnit, L. Chen, Z. Xie, H. Bou-Ammar, and M. Yuan. ShortCircuit: AlphaZero-driven circuit design. *arXiv:2408.09858*, 2024.

- [15] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *IEEE International Conference on Quantum Computing and Engineering*, pp. 824–835, 2024.
- [16] J. Riu, J. Nogu  , G. Vilaplana, A. Garcia-Saez, and M. P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.
- [17] M. Machiya, M. Menickelly, P. Hovland, and J. Liu. MonteQ: A Monte Carlo tree search based quantum circuit synthesis framework. *arXiv:2604.19029*, 2026.